Software Architecture Overview

IndustryApp – post-accident repair offer platform · PA1

Authors: Ömer Efe Öztürk, Bahadır İhsan Herdem · **Course:** BIL 482 Design Patterns · **Version:** 1.0 · **Date:** 05/10/2026

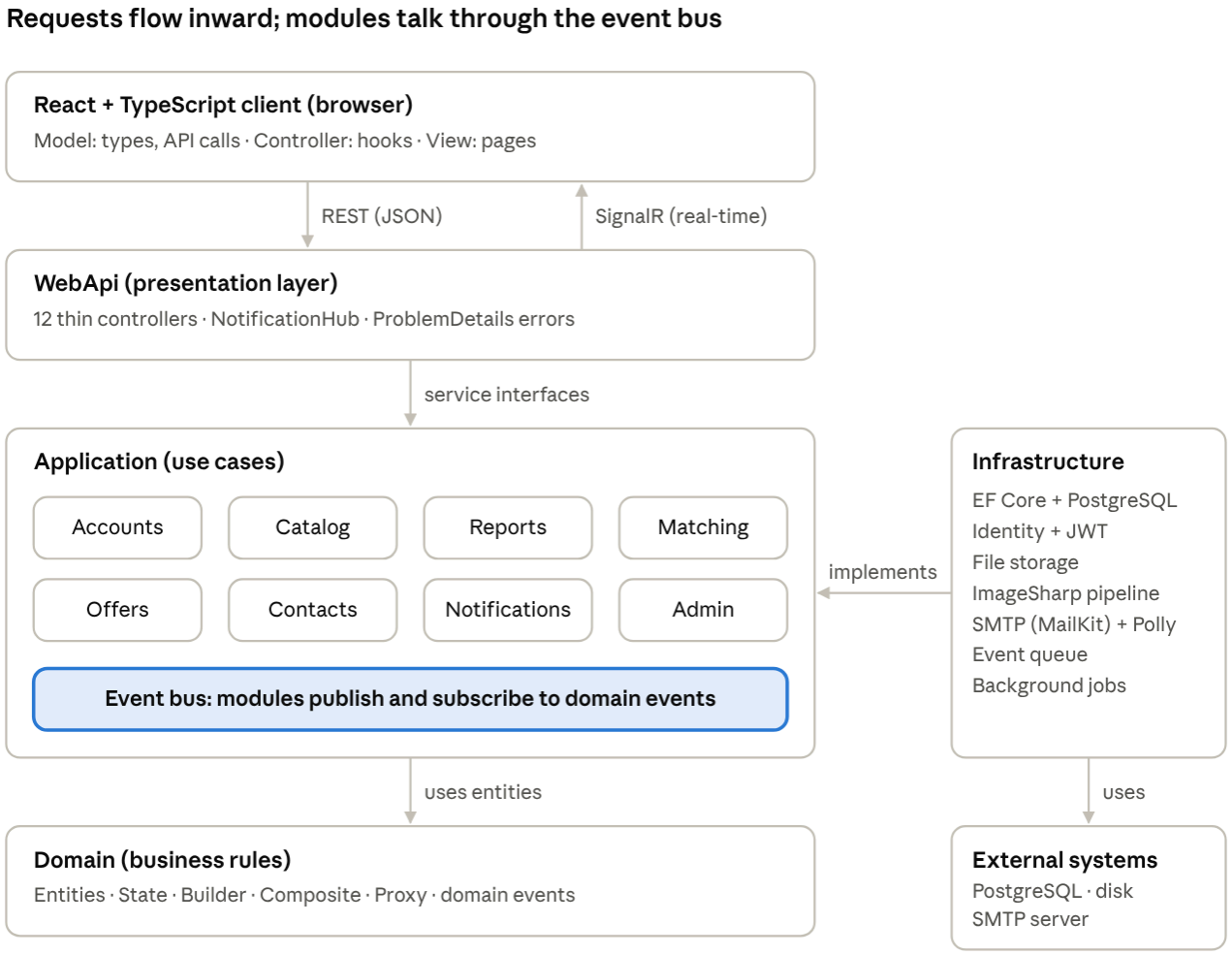
Table of Contents

1. Document Task Matrix
2. Architecture Diagram
3. Architectural Style and Key Decisions
4. Layer Responsibilities
5. Modules and Event Flow
6. How the Architecture Realizes the Use Cases
7. Deployment View

1. Document Task Matrix

#	Task in creating this document	Responsible	Reviewer
T1	Architecture diagram	Bahadır İhsan Herdem	Ömer Efe Öztürk
T2	Architectural style and key decisions	Bahadır İhsan Herdem	Ömer Efe Öztürk
T3	Layer responsibilities and project references	Ömer Efe Öztürk	Bahadır İhsan Herdem
T4	Modules and event flow	Ömer Efe Öztürk	Bahadır İhsan Herdem
T5	Use case realization (UC-01 … UC-04)	Bahadır İhsan Herdem	Ömer Efe Öztürk
T6	Deployment view	Ömer Efe Öztürk	Bahadır İhsan Herdem
T7	Final review, formatting, GitHub upload	Ömer Efe Öztürk	Bahadır İhsan Herdem

2. Architecture Diagram



IndustryApp layers · 4 layers, 8 modules, 1 event bus

The client talks to the WebApi over REST and receives live updates over SignalR. WebApi calls only Application service interfaces; Infrastructure implements those interfaces from outside, and Domain depends on no other layer.

3. Architectural Style and Key Decisions

IndustryApp is a **client-server, layered modular monolith following Clean Architecture**. Between modules it uses **publish-subscribe** through an in-process event bus, and on the client a **distributed MVC** (Model = types and API calls, Controller = React hooks, View = pages and components). Dependencies always point inward: WebApi → Application → Domain; Infrastructure implements the interfaces that Application defines.

Key decisions:

- **K01 – Modular monolith, Clean Architecture:** a two-person team needs one deployable unit and single-transaction consistency; because modules talk through events, they can later be split into services.
- **K02 – In-process event bus (publish–subscribe):** the Reports module does not know about Matching; adding a new listener changes no existing code.
- **K03 / K04 / K05 – .NET 8 Web API, React + TypeScript, PostgreSQL 16:** team experience; API-first for a future mobile client; PostgreSQL for partial unique indexes, JSON and geographic queries.
- **K06 – Sealed offers:** a mechanic sees only their own offer; the rule is applied in the backend query and the response shape.
- **K07 – No seed data:** reference data by versioned SQL, transaction data only from screens; the system is production-ready.
- **K08 – Mechanic approval:** new mechanic accounts start as PendingApproval and do not join matching until an admin approves.
- **K09 – Notification channels:** in-app (SignalR) and e-mail (SMTP); SMS waits behind an Adapter.
- **K10 – File storage:** server disk behind `IFileStorage`; S3/MinIO is one class away.
- **K11 – Authentication:** ASP.NET Core Identity + JWT access token + refresh token, shared by web and mobile.
- **K12 / K13 – Contracts and controllers:** every service has an `I`-prefixed interface; one thin controller per resource; controllers never call each other.

MediatR is deliberately not used; the event dispatcher is the team's own code, so the Observer/Mediator idea can be shown in the project's own classes.

4. Layer Responsibilities

- **Domain** (`IndustryApp.Domain`, references nothing): entities, business rules, the report state machine, domain events, value objects, pattern classes (State, Builder, Composite, price Decorator, Proxy). Does not write to the database, does not know HTTP, does not call `DateTime.UtcNow` (time comes in as a parameter, so time windows are testable).
- **Application** (`IndustryApp.Application`, references Domain): orchestrates use cases — authorization and ownership checks, validation, loading through repository interfaces, calling domain methods, saving, publishing events. Holds the service, repository and infrastructure interfaces and the Factory, Strategy, Command and Chain of Responsibility classes. Writes no SQL and knows nothing about the file system, SignalR or SMTP.

- **Infrastructure** (`IndustryApp.Infrastructure` , references Application): EF Core + PostgreSQL, Identity, repositories and Unit of Work, file storage, image processing, e-mail, the event queue, background jobs, resilience pipelines. Contains no business rules.
- **WebApi** (`IndustryApp.WebApi` , references Application and Infrastructure only for DI registration): receives HTTP requests, binds them to request models, calls one service interface, maps the result to an HTTP status; hosts the SignalR hub and global error handling. Contains no business logic and never uses the DbContext.
- **Client** (React + TypeScript): Model (types, API calls), Controller (hooks for data, forms and real-time events), View (pages and components). It reaches the backend only through REST and the SignalR hub.
- **Tests:** `UnitTests` reference Domain and Application and run without a database; `IntegrationTests` run the WebApi against a real PostgreSQL container.

5. Modules and Event Flow

The Application layer is split into eight modules — Accounts, Catalog, Reports, Matching, Offers, Contacts, Notifications, Admin — and each module has a folder of the same name in every layer, so the code of a feature is always in a predictable place. A module that needs another module either injects its service interface or, preferably, publishes an event.

How an event travels:

1. A domain method (e.g. `AccidentReport.SelectOffer`) records the event inside the entity.
2. `UnitOfWork.SaveChangesAsync` collects the events from changed entities and saves everything in one transaction.
3. Only if the save succeeds, `InProcessEventDispatcher` writes the events to an in-memory channel and the HTTP response returns immediately.
4. `EventProcessingBackgroundService` reads the channel, opens a new DI scope per event and runs every registered `IEventHandler<T>` .
5. If one handler fails, the error is logged and the other handlers still run.

Main events and listeners:

- `AccidentReportCreatedEvent` (Reports) → Matching starts automatic matching.
- `MechanicsMatchedEvent` (Matching) → Notifications sends "new job" to matched mechanics, or "no mechanic found" to the customer.

- `OfferSubmittedEvent`, `OfferUpdatedEvent`, `OfferWithdrawnEvent` (Offers) → Notifications informs the customer; the real-time handler updates the open report screen.
- `OfferSelectedEvent`, `OfferSelectionUndoneEvent` (Offers) → the winning and the other mechanics are notified.
- `ReportCompletedEvent`, `ReportCancelledEvent`, `ReportExpiredEvent` (Reports, expiration job) → the affected users are notified.
- `MechanicApprovedEvent`, `MechanicRejectedEvent` (Admin) → the mechanic receives an in-app notification and an e-mail.

Because events are published after the commit, a user who opens a notification never sees a record that was not saved; because handlers run in the background, report creation does not wait for matching or e-mail.

6. How the Architecture Realizes the Use Cases

Every use case follows the same request path — view → hook → REST endpoint → thin controller → Application service interface → domain objects → Unit of Work → events — so only the modules involved change from case to case.

- **UC-01 Registration and login:** `AuthController` calls `IAuthService.RegisterAsync`, which picks the profile factory for the requested role (Factory Method) and creates the user, profile and consents in one transaction; Identity and JWT live in Infrastructure behind `IIdentityService` and `IJwtTokenService`. After login, `AccountService` chooses the dashboard factory for the role (Abstract Factory) and the client draws its menu from that response. Authorization policies (CustomerOnly, ApprovedMechanic, AdminOnly) guard every endpoint.
- **UC-02 Create accident report:** photos go to `PhotosController` first; the image-processing chain (Decorator) runs in Infrastructure and the file is saved through `IFileStorage` (Adapter), so Application never touches the disk. `IReportService.CreateAsync` assembles the report with `AccidentReportBuilder` from the wizard data and the damage tree (Composite), saves it, and the commit publishes `AccidentReportCreatedEvent`. The response returns before matching starts.
- **UC-03 Job notification and offer:** the Matching module listens to the creation event, finds candidates with the configured `IMatchingStrategy` and narrows them with the `IMechanicFilter` chain, then publishes `MechanicsMatchedEvent`. Notifications delivers in-app messages through `IRealtimePublisher` (SignalR) and e-mails through

`INotificationChannel` (Adapter) wrapped by the resilient channel (Circuit Breaker + Retry). The mechanic's offer is priced by the Decorator chain in Domain and pushed live to the customer.

- **UC-04 Offer selection and contact:** `OfferSelectionController` calls `IOfferSelectionService`, which checks ownership, creates `SelectOfferCommand` and executes it; the report's current State object decides whether the transition is allowed. One transaction saves the new statuses and a snapshot for undo, and a partial unique index stops a second accepted offer at database level. `ContactService` wraps the real contact data in `ContactInfoProxy`, so the phone number is revealed only to the matched pair. Offer sorting uses an `IOfferSortStrategy` selected by the request.

7. Deployment View

Production will run on one Linux server (2 vCPU, 4 GB RAM) as three Docker Compose containers:

- **web** — Caddy: serves the built React files, terminates HTTPS with automatic Let's Encrypt certificates, and forwards `/api` and `/hubs` to the API. One domain for everything removes CORS and cookie issues.
- **api** — the ASP.NET Core application (REST API, SignalR hub, background jobs), running as a non-root user; migrations run as a separate bundle at deploy time, never on start-up.
- **db** — PostgreSQL 16 with its data in a Docker volume; not exposed outside the server.

Uploaded photos live in a separate volume. Only ports 22, 80 and 443 are open. A nightly job dumps the database and archives the uploads, keeping 7 days. `/health` reports database connectivity and is used by the Docker health check. In development only the database runs in Docker; the API runs with `dotnet watch` and the client with the Vite dev server, which proxies `/api` and `/hubs`.